00_frontmatter

Helios — Implementation Playbook

`IMPLEMENTATION_PLAYBOOK.md` · The standalone build manual · **Version:** v1.0 · **Date:** 2026-06-03

> **Read this first.** This manual assumes you have **no further AI assistance** — just this repo and your own hands. Everything you need is already here. The production-grade *code and specs already exist in the docs* (full dbt SQL in `DBT_GUIDE.md`, MCP/agent skeletons in the architecture docs, the live `semantic_layer.yaml`, the 50 `eval/scenarios/*.yaml`). This playbook's job is to **sequence the build** and give you, for each milestone, the **objective, the exact files, the commands, the tests, the success criteria, and the mistakes to avoid**. Build the milestones in order. Never skip the golden tests on M3 — they fail *silently*.

0.1 What you are building (30-second recap)

Helios is an autonomous growth-diagnosis engine on the GA4 Google Merchandise Store dataset. Raw GA4 events → **dbt** marts → a **governed semantic layer** → **5 MCP servers** (the only paths to SQL and to math) → **7 agents** on a deterministic state machine that diagnose *why* the funnel moved (mix-shift vs rate-change), price it in dollars, and ship a Decision Brief — graded by a **50-scenario offline benchmark** (target ≥85% root-cause accuracy vs ≤45% naive baseline).

0.2 Prerequisites & toolchain (do this once)

Need	What
Cloud	A **GCP project** with billing; BigQuery API enabled; read access to `bigquery-public-data` (public, no setup).
Auth	A least-privilege service account (`roles/bigquery.dataViewer` + `roles/bigquery.jobUser`) **or** `gcloud auth application-default login` for dev.
Local	**Python 3.11**, the **gcloud SDK**, **git**, and (optionally) the **GitHub CLI** for CI.

Need	What
LLM	An **Anthropic API key** (`ANTHROPIC_API_KEY`) for the Claude Agent SDK — or any tool-calling LLM behind the same MCP interface (see §0.6).

`requirements.txt` (create at repo root in M0):

```
dbt-core≥1.7,<2.0
dbt-bigquery≥1.7,<2.0
google-cloud-bigquery≥3.0
scipy≥1.11
statsmodels≥0.14
prophet≥1.1            # forecasting (or swap pmdarima)
pandas≥2.0
pyyaml≥6.0
mcp≥1.0                # Model Context Protocol SDK
anthropic≥0.39         # or your LLM provider's SDK
pytest≥8.0
```

dbt packages (via `packages.yml`, installed with `dbt deps`): `dbt_utils`, `dbt_expectations`, `dbt_project_evaluator`, `codegen`.

Environment variables (set in your shell / CI secrets):

```
export GOOGLE_APPLICATION_CREDENTIALS=/path/to/sa.json   # or use ADC
export HELIOS_MM_TOKEN=...             # warehouse-mcp bearer (M6)
export ANTHROPIC_API_KEY=...           # agents (M7)
export HELIOS_VECTOR_STORE_URI=...     # memory ANN recall (M8)
```

> Windows note: activate the venv with `.venv\Scripts\activate` (POSIX: `source .venv/bin/activate`). All dbt/gcloud/python commands are otherwise identical.

0.3 How each milestone is written

Every milestone below has the same six parts: **Objective · Files to create · Commands to run · Tests to run · Success criteria · Common mistakes**. "Files to create" names the path **and the doc + section to copy the code from** — you are assembling, not inventing. Files marked *(already exists)* must **not** be regenerated.

0.4 Global build map

Milestone	Builds	Level	Exit gate (one line)
M0	Repo + GCP/IAM + dbt config	L1	`dbt debug` green; bounded query over `events_*` returns rows
M1	Sources, macros, seed	L1	`dbt deps` + `dbt seed` ; macros compile
M2	Staging models	L1	`dbt build --select staging` ; key tests green
M3 ★	Sessionization + funnel keystone	L1	`session_key` unique; **funnel monotonicity** test passes
M4	Marts (core/finance/growth)	L1	`dbt build` green; revenue reconciles to the cent; channels = 10
M5	Semantic layer live	L1/L2	`validate_semantic.py` → 0 dangling refs against real marts
M6	semantic-mcp + warehouse-mcp	L1	`build_query→dry_run→run_query→reconcile` round-trips; budget caps
M6b	stats / experiment / report MCP	L1/L2	`decompose_change` golden test passes
M7	Minimal autonomous loop	**L1 DONE**	anomaly → brief in <5 min; 0 hallucinated columns
M8	Memory store	L2	`save_diagnosis` → `recall_prior` round-trips; calendars seeded
M9	Full 7-agent loop	L2	PASS findings carry significance + $ + experiment
M10	Eval harness + CI	**L2 DONE**	≥85% vs ≤45%; hallucination = 0; CI gate green
M11	Autonomy & depth	**L3 DONE**	<5 min/run on schedule; accuracy holds all dims
M12	Productionization & frontier	—	(deferred) multi-tenant; causal inference

0.5 Target file tree (check each off as you build it)

```
helios/
├─ requirements.txt · .gitignore · dbt_project.yml · profiles.yml · packages.yml · mcp_servers.yaml
├─ scripts/validate_semantic.py
├─ models/
```

```
│   ├─ staging/   src_ga4.yml · stg_ga4__events.sql · stg_ga4__event_params.sql · stg_ga4__schema.yml
│   ├─ intermediate/ int_ga4__sessionized.sql · int_ga4__funnel_steps.sql · int_ga4__schema.yml
│   ├─ marts/core/ fct_sessions · fct_funnel · fct_daily_funnel · dim_users · dim_items · dim_channel
│   ├─ marts/finance/ fct_orders · fct_order_items (+finance__schema.yml)
│   ├─ marts/growth/ fct_funnel_by_dim · fct_cohorts (+growth__schema.yml)
│   └─ semantic/  semantic_layer.yaml (exists) · metrics__schema.yml
├─ macros/ get_event_param.sql · sessionize.sql · channel_group.sql · test_revenue_reconciles.sql
├─ seeds/ channel_group_mapping.csv  · snapshots/ snap_dim_items.sql  · tests/ assert_*.sql
├─ sql/ helios_memory_ddl.sql
├─ helios/
│   ├─ mcp/ base.py · schemas.py · warehouse.py · semantic.py · stats.py · experiment.py · report.py
│   ├─ agents/ framework.py · orchestrator.py · monitor.py · decompose.py · diagnose.py · critic.py ·
│   ├─ runner.py
│   └─ eval/ injector.py · runner.py · report.py · scorers/*.py · scenarios/*.yaml (exist)
├─ eval/ gates.yaml · baselines/main.json
├─ .github/workflows/ ci.yml
└─ docs/  (the specs you build FROM — already exist)
```

0.6 Conventions you must obey (cheat-sheet)

- `session_key = TO_HEX(MD5(CONCAT(user_pseudo_id,'-',CAST(ga_session_id AS STRING))))` ; one expression everywhere.
- `reached_*` funnel flags are **max-downstream monotonic** → step rates ≤ 1 by construction (`did_*` is retired).
- Exactly **10** channel groups, defined in **one** macro `channel_group_case()` ; `traffic_source` is user first-touch — prefer session-scoped `event_params` source/medium.
- Money: `*_in_usd` only; rates `SUM(num)/SUM(den)` after grouping.
- The LLM **never** writes SQL (only `semantic-mcp`) and **never** computes stats (only `stats-mcp`); `dry_run` before every `run_query` .
- Stats are seeded (`rng_seed = 1729`); per-run byte budget ≤ 5 GiB.

If you can't use the Anthropic API: the analytics value (M0–M5: governed marts + the semantic layer) is fully usable with no LLM at all. For the agents (M7+), the MCP servers are LLM-agnostic — point any tool-calling model at the same `mcp_servers.yaml` interface; only `helios/agents/framework.py` (the model client) changes.